Libras: A Fair, Secure, Verifiable, and Scalable Outsourcing Computation Scheme Based on Blockchain

Lijuan Huo, Libing Wu, Zhuangzhuang Zhang, Chunshuo Li, Debiao He, *Member, IEEE*, and Jing Wang

***Abstract*— Existing multitask outsourcing computations struggle to guarantee the fairness for participants and the correctness of the computation results. Some solutions use blockchain to address the fairness issue in outsourcing computations. However, blockchain suffers from poor data privacy due to its public and transparent nature, as well as the latency because of limited scalability. To effectively confront these problems, we propose the Libras: a fair, secure, verifiable and scalable outsourcing computation scheme based on blockchain. In Libras, tasks are divided into multiple sub-task blocks, coupled with a deposit mechanism that enforces fairness throughout the process. Libras integrates a commitment mechanism with on-chain and off-chain collaboration for security, where the computation results are securely stored off-chain while proofs of these results are immutably recorded on-chain. Moreover, it employs a Directed Acyclic Graph (DAG)-based ledger architecture to significantly expedite transaction confirmations and facilitate elastic scalability. Furthermore, we devise a batch verification algorithm to simultaneously verify the accuracy of all computation results. Theoretical analysis and experiments demonstrate that Libras is fair, secure, verifiable, and scalable. The comparison results indicate that the verification time is 1.2× that of FVP-EOC.**

***Index Terms*— Verifiable computing, blockchain, outsourcing computation, vector commitment.**

Manuscript received 20 October 2023; revised 27 March 2024; accepted 9 May 2024. Date of publication 20 May 2024; date of current version 28 May 2024. This work was supported in part by the National Key Research and Development Program of China under Grant 2021YFB3101100; in part by the National Natural Science Foundation of China under Grant U20A20177, Grant 62272348, Grant U22B2022, and Grant 62202197; in part by Wuhan Science and Technology Joint Project for Building a Strong Transportation Country under Grant 2023-2-7; and in part by the Open Research Fund from Guangdong Laboratory of Artificial Intelligence and Digital Economy (SZ) under Grant GML-KF-22-07. The associate editor coordinating the review of this manuscript and approving it for publication was Dr. Alptekin Küpçü. *(Corresponding authors: Libing Wu; Zhuangzhuang Zhang.)*

Lijuan Huo, Libing Wu, and Zhuangzhuang Zhang are with the Key Laboratory of Aerospace Information Security and Trusted Computing, Ministry of Education, School of Cyber Science and Engineering, Wuhan University, Wuhan 430072, China, and also with Guangdong Laboratory of Artificial Intelligence and Digital Economy (SZ), Shenzhen 511442, China (e-mail: lijuanHuo@whu.edu.cn; wu@whu.edu.cn; zhzhuangzhuang@whu.edu.cn).

Chunshuo Li is with the Key Laboratory of Aerospace Information Security and Trusted Computing, Ministry of Education, School of Cyber Science and Engineering, Wuhan University, Wuhan 430072, China (e-mail: 2018302180127@whu.edu.cn).

Debiao He is with the Key Laboratory of Cryptography of Zhejiang Province, Hangzhou Normal University, Hangzhou 311121, China, and also with the School of Cyber Science and Engineering, Wuhan University, Wuhan 430072, China (e-mail: hedebiao@163.com).

Jing Wang is with the School of Cyber Science and Engineering, Huazhong University of Science and Technology, Wuhan 430074, China (e-mail: cswjing@hust.edu.cn).

Digital Object Identifier 10.1109/TIFS.2024.3403489

I. Introduction

EDGE computing is a paradigm of distributed computing that situates data storage and computation at the edge nodes of the network [1]. End devices are usually lightweight for portability in edge computing scenarios such as smart medical [2], intelligent transportation [3], and energy system [4]. They cannot provide adequate capacity to allocate storage and computing resources. However, in practical applications, end devices are required to analyze large amounts of data quickly and make accurate responses. Hence it is difficult for them to fulfill the requirements [5] due to the limited resources and computing power.

The burgeoning advancements in edge computing technology have ushered in an extensive array of outsourced services that encompass storage and computations. End devices can outsource their multifarious data onto edge nodes for complex computing. While outsourcing computation can indeed provide users with a viable means to overcome the challenges stemming from limited storage and computational resources, the attendant issues deserve to be taken seriously [6], [7], [8]. For example, most existing solutions tend to neglect the aspect of fairness in outsourcing services, primarily evident in unevenly distributed computing resources and inadequate incentive and punishment mechanisms. Specifically, Certain devices, particularly those with weak computing power, may consistently be overlooked when assigning computational tasks. In addition, the server successfully accomplishes the task, yet the user defaults on the payment. The process of outsourcing computation is untraceable, rendering it implausible to accurately identify the defaulting user and impose suitable punitive measures.

Fortunately, blockchain has garnered significant interest from the industrial Internet of Things (IIoT) because of its decentralization, transparency, security, and immutability [9], [10]. Blockchain's inherent features offer innovative solutions to tackle the challenge of fairness in outsourcing computations. More specifically, it enforces fairness through an incentive and punishment mechanism. Before any transaction commences, both users and servers are required to place a collateral deposit. If either party acts in bad faith or deviates from the agreed terms, their deposit is forfeited to safeguard the rights and interests of the other participants. In addition, blockchain, besides being a supervisor of users and servers, is also a verifier. Beyond overseeing user-server interactions, it undertakes the crucial responsibility of verifying the computation results of outsourced tasks.

1556-6021 © 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Institute of Information EngineeringCAS. Downloaded on March 28,2026 at 14:48:22 UTC from IEEE Xplore. Restrictions apply.

While blockchain undeniably offers notable benefits in resolving fairness concerns in multitasking outsourced computing, it still confronts several issues that need to be addressed. Considering the diverse complexities of computational tasks and the varying capacities among servers, how to design a fair outsourcing solution based on blockchain to protect the interests of all stakeholders. Moreover, due to blockchain's public and transparent nature, data is openly accessible, which could compromise the interests of the users and hinder the successful implementation of outsourcing tasks in the absence of adequate protection measures. Consequently, how to safeguard the security of outsourcing computation outcomes is an inevitable challenge. Furthermore, since implementing an outsourcing solution in the blockchain network puts all participants in an environment where they do not trust each other, how to achieve efficient and reliable verification is an unavoidable issue to be tackled. Lastly, the decentralized nature of blockchain poses constraints like limited scalability [11] and increased latency. It is not suitable for edge outsourcing scenarios that require high real-time performance [12]. Hence, how to improve performance in outsourcing computation based on blockchain will be a significant problem.

To simultaneously solve the above challenges of fairness, security, verifiability, and scalability, we propose the Libras: a fair, secure, verifiable and scalable outsourcing computation scheme based on blockchain. Libras consists of three primary stages, namely Task Allocation, Task Calculation, and Result Verification. Initially, Libras implements a task allocation policy to divide complex tasks into multiple sub-tasks and pay a deposit when servers sign a contract to perform the tasks. Recognizing the security vulnerabilities in directly storing raw computation results on the blockchain, upon the culmination of tasks, servers opt to maintain these computation results off-chain for security. Simultaneously, servers store proofs of results on-chain in the shape of a directed acyclic graph (DAG) [13] ledger structure. Ultimately, Libras employs an advanced batch verification algorithm grounded in a commitment mechanism to efficiently and reliably verify whether the task has been executed accurately. Our contributions are as follows.

(1) We design a task allocation policy to guarantee fairness in Libras. Multiple tasks are ranked in ascending order based on their deposit and divided into smaller chunks by the Euclidean algorithm, allowing even weaker servers to have the opportunity to select tasks and avoiding the idleness of computational resources.

(2) We integrate a commitment mechanism with on-chain and off-chain collaboration in blockchain. In the Libras, we store computation results off-chain and store proofs on-chain for security. Moreover, we utilize a DAG-based ledger structure to provide fast transaction confirmation and elastic scalability.

(3) We propose a batch verification algorithm to simultaneously verify whether all computation results of the outsourced tasks are accurate. In the verification process, the basis function of each server is computed using Lagrange interpolation based on basis functions, and then the computation results under the same task are verified in batch by the proofs aggregation.

(4) Theoretical analysis and experimental results manifest that the Libras is fair, secure, verifiable, and scalable. Furthermore, we simulate outsourcing computations for matrix multiplication and modular exponentiation, comparing our approach against a similar scheme FVP-EOC [14], demonstrating that the verification time of our batch verification algorithm is $1.2\times$ that of FVP-EOC.

The remainder of this paper is structured as follows. Section II presents a review of relevant literature. Section III depicts the preliminaries. In Section IV, we give a detailed description of the Libras: a fair, secure, verifiable, and scalable outsourcing computation scheme based on blockchain in this paper. In Section V and Section VI, we conduct a performance evaluation of the Libras and security analysis. Finally, Section VII offers concluding remarks on the paper.

II. Related Work

A. Outsourcing Computation in Edge Computing

In traditional outsourcing computation, users delegate sophisticated tasks to powerful centralized cloud servers, though this practice often leads to elongated computation times. Seeking to optimize this process, some works design specific outsourcing strategies for different computational tasks, such as matrix multiplication [15], [16], modular exponentiation [17], [18], [19], polynomial function evaluation [20], and polynomial multiplication [21], and delegate the tasks to multiple edge servers. To take full advantage of the computational resources, tasks need to be allocated fairly and efficiently. Some researchers have utilized the greedy algorithm [22], the minimum execution time algorithm [23], the ant colony algorithm [23], the firefly algorithm [24], and a task bidding strategy [14] to delegate tasks to servers for efficient execution. However, these task allocation methods are complex and time-consuming, and may induce a heavy computation burden. By resorting to outsourcing computation in the realm of edge computing, multiple edge servers, each equipped with less storage and computational power compared to a centralized cloud server, collaborate to execute tasks, thereby significantly reducing the cumulative computation time. Moreover, alongside efficient task delegation, the verification of computation results plays a pivotal role in ensuring the credibility of the outsourcing process. Cai et al. [25] proposed OVERSEE, which verifies whether the task has been computed accurately utilizing the Report-Proof and Sampling-Challenging. Chen et al. [26] designed a privacy-preserving and verifiable outsourcing approach. This approach safeguards the confidentiality of data related to computational tasks and ensures the validity of the computation results. Li et al. [27] elaborated a distributed and secure system DSOS. In DSOS, the collaborative verification among the edge servers guarantees the accuracy of the outcomes. Although these schemes can effectively guarantee the security of data and the accuracy of computations, they generally overlook the issue of ensuring fair payment for outsourced services. Moreover, there will be some servers with weak computing power that cannot be assigned to tasks, resulting in a waste of computing resources.

Authorized licensed use limited to: Institute of Information EngineeringCAS. Downloaded on March 28,2026 at 14:48:22 UTC from IEEE Xplore. Restrictions apply.

TABLE I
A COMPARISON OF DIFFERENT SOLUTIONS

Scheme	fairness	security	verifiability	scalability	Security model		
					U	S	V
Dredas [32]	×	✓	✓	×	S-H	S-H	S-H
Huang et al. [33]	optimistic	×	✓	×	S-H	S-H	S-H
Zhang et al. [7]	robust	✓	✓	×	M	M	S-H
OBFP [34]	robust	✓	✓	×	M	M	S-H
FVP-EOC [14]	optimistic	✓	✓	×	H	M	S-H
Libras	optimistic	✓	✓	✓	M	M	S-H

✓ and × indicate supported and unsupported features. U, S and V represent user, server and verifier respectively. H, S-H and M denote honest, semi-honest and malicious entities, respectively. In addition, optimistic fairness means that a trusted third party (TTP) is still required in the event of a dispute, and robust fairness represents that fairness can be achieved without a TTP, but it will somewhat compromise the fairness of workers.

B. Outsourcing Computation in Blockchain

Owing to the lack of trust between users and servers, conventional payment methods struggle to guarantee fairness. In scenarios where users remunerate servers in advance, there is no assurance that the servers will execute the computation task appropriately. Conversely, if the servers first undertake the computational task and then deliver the results to the users, it is uncertain that users will adhere to their agreed-upon payment. Fortunately, there has been an increasing amount of research using blockchain to address the payment fairness issues in outsourcing computation. Cui et al. [28] designed a payable outsourced decryption scheme based on blockchain, which enables users to compensate a server once it successfully carries out the delegated decryption task. However, this scheme involves complex cryptographic algorithms when making payments. Subsequently, some research further introduced smart contracts to blockchain-based fair payments of bilinear pairings [6], [29], linear regression [8], and polynomial computation [30]. In addition, Chen et al. [31] introduced an incentive-compatible outsourcing computation solution and gave rigorous evidence that rational participants lack the incentive to stray from honest conduct when the overhead for delegation is minimal. These schemes are implemented on a public blockchain platform that supports smart contracts, for instance, Ethereum. Generally, this brings long transaction confirmation times and limited scalability, hindering practical applications.

C. Outsourcing Computation in Edge Computing Based Blockchain

While blockchain offers notable benefits in solving the fairness of outsourcing computation, the users initiating outsourcing tasks are generally weak in computing power or even do not have mining capability. Therefore, in practical applications, the participation of edge servers with strong computational power and storage resources is necessary. Currently, some scholars have researched outsourcing computation in edge computing based on blockchain. Fan et al. [32] elaborated the Dredas, a decentralization auditing system that utilizes Ethereum's smart contract to validate the outsourced data held on servers. Dredas incorporates advanced technologies such as the latest nonce, BLS signature, and bilinear pairing to provide robust data auditing security. However, Dredas does not propose a specific strategy to safeguard fairness. Huang et al. [33] used the commitment-based sampling strategy to construct a fair Bitcoin-based payment strategy for outsourcing computation. But this scheme requires a third party to address the trust issue, and thus the fairness in this scheme is only positive rather than robust, potentially making the outsourced data insecure. Later, Zhang et al. [7] improved the scheme of Huang et al. and introduced BPay, to achieve robust fairness and accomplish fair compensation for outsourcing tasks devoid of any trusted intermediary. However, the drawback of these schemes is that participants are subjected to a substantial computational burden due to the employment of zero-knowledge proof (ZKP). Lin et al. [34] constructed an OBFP system free from ZKP that leverages blockchain, cryptographic accumulator, secure commitment, and so on, to mitigate these limitations of unfairness and significant computation costs. The proposed system can satisfy robust fairness and does not involve any TTP during the process of trading, but somewhat compromises the worker's fairness. Li et al. [14] proposed a blockchain-based edge outsourcing computation strategy that is fair, verifiable, and privacy-preserving (FVP-EOC). It is a typical blockchain-based outsourcing computing solution that simulates the outsourcing computations of matrix multiplication and modular exponentiation. FVP-EOC and Libras rely on blockchain nodes for verification and achieve optimistic fairness. In FVP-EOC, the user is trusted while the server is considered malicious. In contrast, in [7] and [34] and Libras, neither the users nor the servers are fully trusted and can potentially exhibit malicious behavior driven by self-interests. Verifiers in these schemes are semi-honest. However, most existing solutions based on blockchain suffer from high time latency because each block generation takes about 10 minutes. Moreover, the requirement to log data into the blockchain could potentially impose tremendous storage demands and thus a massive gas cost. In short, none of these solutions effectively solve the scalability issue. A comparison of these solutions is shown in Table I.

III. PRELIMINARIES

In this section, we first illustrate the cryptographic building blocks required for Libras, followed by a detailed explanation of the system model, adversary models, and security goals of Libras.

Authorized licensed use limited to: Institute of Information EngineeringCAS. Downloaded on March 28,2026 at 14:48:22 UTC from IEEE Xplore. Restrictions apply.

A. Cryptographic Building Blocks

1) Lagrange Interpolation Based on Basis Functions: Lagrange interpolation [35] based on basis functions is a method to construct a polynomial $y = f(x)$ that best fits a group of points (x_1, y_1), $(x_2, y_2), \ldots, (x_n, y_n)$. Some scenarios of outsourcing computation require that the value of y_i computed from x_i is not shared among servers. In such cases, Lagrange interpolation based on basis functions can meet this requirement. The construction of the polynomial can be divided into the following steps.

1) Given a collection of specific points, represented as $(x_i, y_i)_{i=1}^{n}$, where each x_i denotes an independent variable and each corresponding y_i signifies the dependent variable.
2) Define the basis function I_i which satisfies $I_i = \prod_{i \neq j}^{1<j<k} \frac{x-x_j}{x_i-x_j}$.
3) Construct the Lagrange interpolation polynomial $f(x) = \sum_{i=1}^{n} y_i I_i$

2) Secret Sharing: In secret sharing [36], the secret s is divided into n parts, and each part is referred to as a sub-secret and held by a distinct holder. The secret s can be reassembled when k or more participants combine their sub-secrets. Conversely, if the number of participants with sub-secrets is less than k, the reconstruction of s is not achievable and no one can gain any insight about it. The secret to be shared is encoded as a constant term a_0 in a polynomial $f(x) = a_0 + a_1 x + a_2 x^2 + \ldots + a_{k-1} x^{k-1}$ of order $k-1$.

1) Split the secret: randomly generate coefficient values from a_1 to a_{k-1}. On this polynomial $f(x)$ of order $k-1$, take any k different points (x_1, y_1), (x_2, y_2), $\ldots$, (x_k, y_k), and assign them to the participants. Each participant thus gets a secret share.
2) Recover the secret: k participants aggregate the secret shares together and then substitute k points into the original function to determine a unique polynomial. That is, it determines the value of $a_0, a_1, a_2, \ldots, a_{k-1}$, where a_0 is the secret s. And over any $k-1$ points, there are theoretically an infinite number of curves in the real number field that do not leak out any useful information about a_0.

Secret sharing does not require centralized key management and is suitable for different distributed system scenarios such as blockchain. Unlike encryption, each participant in secret sharing holds a separate piece of information rather than a key that needs to be protected. This avoids potential security vulnerabilities in key distribution and management. Even if some of the shares are lost or accessed by a malicious attacker, the secret will not be disclosed as long as the legitimate shares above the threshold remain safe. For secret sharing, the process of generating and recovering secrets usually involves only simple mathematical operations such as addition and multiplication, especially in secret sharing based on linear algebra. Compared to complex symmetric or asymmetric encryption algorithms, this may mean a lower computational cost in distributed scenarios.

3) Bilinear Pairings: Let $(\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T)$ be a bi-linear group of prime order p, whose generators are g_1, g_2, $g_T = e(g_1, g_2)$ respectively. $\forall g_1 \in \mathbb{G}_1, g_2 \in \mathbb{G}_2$ and $a, b \in \mathbb{Z}_p$, there is $e(g_1{}^a, g_2{}^b) = e(g_1, g_2)^{ab}$. A useful property of $e(\cdot, \cdot)$ is that $e(g_1{}^a, g_2{}^b)e(g_1{}^c, g_2{}^d) = e(g_1, g_2)^{ab+cd}$, where $a, b, c, d \in \mathbb{Z}_p$.

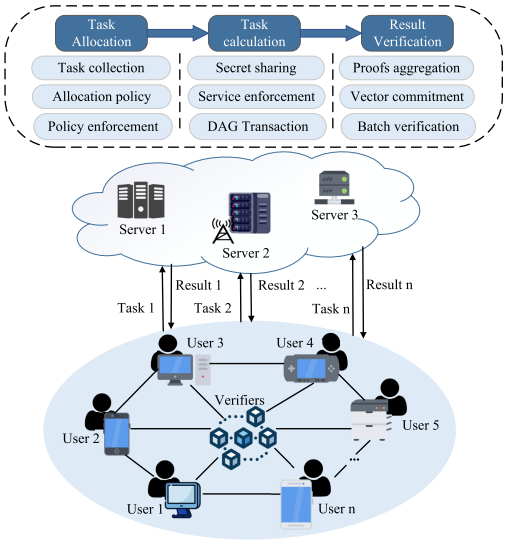


Fig. 1. The framework of Libras.

4) Commitment: A Vector Commitment (VC) [37] is a scheme that allows a participant to commit to a vector (i.e., a set of values) with the assurance that the participant can later prove that the commitment is correct without disclosing any elements of the vector.

- $pp \longleftarrow setup(1^\lambda, n)$. Generate public parameters that all parties will employ to commit the message vectors of length n.
- $C \longleftarrow commit(\mathbf{a}, i)$. Take as input a set of values $\mathbf{a} = [a_0, a_1, a_2, \ldots, a_{n-1}]$. Output a commitment C.
- $\pi_i \longleftarrow prove(\mathbf{a}, i)$. Output a proof π_i for position i in $\mathbf{a}$.
- $\pi_I \longleftarrow aggregate((a_i, \pi_i)_{i \in I}, I)$. Aggregate individual proofs π_i for values a_i into an aggregated proof π_I.
- $b \longleftarrow verify(C, \pi_I, (a_i)_{i \in I}, I)$. Verify the proof π_I that each position $i \in I$ has value a_i against commitment C.

B. System Model

Fig. 1 depicts the system model of Libras, which encompasses an edge outsourced computing scenario comprising users, servers, and verifiers. The respective roles of each entity are illustrated as follows.

1) **Users:** Users are generally some IoT devices, such as mobile phones, computers, vehicle devices, etc. Due to the limited resources and weak computing power, they are unable to handle complex computational tasks. Therefore, the users initiate the outsourcing computation tasks and outsource their data to cloud servers with strong computing power for computing.

2) **Servers:** Servers are some cloud servers that execute the computational task and deliver the result to users upon completion. To ensure that the user can verify whether the task has been computed accurately, the server needs to compute related proofs. Servers store computation results off-chain and store proofs on-chain for security and privacy.

Authorized licensed use limited to: Institute of Information EngineeringCAS. Downloaded on March 28,2026 at 14:48:22 UTC from IEEE Xplore. Restrictions apply.

3) **Verifiers:** The verifiers in this scenario are a collection of decentralized edge servers deployed on the blockchain network. Their function is to verify whether the computation results returned by servers are accurate. The verifiers reward those servers that submit correct results, while they impose penalties on malicious servers by confiscating their deposits if their computation results fail to pass the verification.

C. Adversary Models

1) **Malicious users:** Malicious users exploit the computational resources offered by servers without bearing the corresponding remuneration. They try to refuse payment, upon receiving the correct computation results from the servers. Therefore, it is necessary to implement a deposit mechanism.

2) **Malicious servers:** Malicious servers may submit incorrect computation results and deceive verifiers to obtain rewards. In addition, in an open and transparent blockchain environment, malicious servers can easily expose users' privacy by uploading computation results in plaintext on the chain. Thus, we need not only to verify whether the task has been computed accurately and detect any malicious entities involved in the computations, but also to protect users' privacy.

3) **Honest but curious verifiers:** Honest but curious verifiers execute the verification process honestly, without engaging in active attacks such as forgery or tampering. However, they attempt to infer additional information from the messages obtained by honestly executing the protocol. This adversary ostensibly adheres to the protocol meticulously while covertly seeking to acquire more information. Hence, the computation results should not be directly delivered to the verifiers. Instead, only the proofs of the computation results are sent to the verifiers for verification.

D. Design Goals

1) **Fairness:** Fairness guarantees equal opportunities for participants and fair recompense for all parties. For servers, fairness means that servers of any computational power can be assigned tasks, even those with weak computational capacity. Furthermore, it ensures that no malicious user can exploit the computational resources offered by servers without bearing the corresponding remuneration. For users, fairness means that dishonest servers are penalized if they do not complete their computational tasks correctly or intentionally submit false computational results.

2) **Security:** The confidentiality of the computation results should be maintained throughout the entire duration, neither accessible to anyone other than the user and the server submitting the computation results, nor directly exposed on the blockchain. Moreover, servers do not share each other's computation results.

3) **Verifiable:** The computation results submitted by servers can be verified for correctness by the task verifiers. While it is crucial to appropriately reward honest servers for their contributions, it is equally important to implement punitive measures against malicious servers.

4) **Scalability:** Unlike the traditional chain structure, DAG-based blockchain can increase transaction throughput, improve transaction processing speed, and further reduce the latency of transmitting computation results.

TABLE II
THE DESCRIPTIONS OF NOTATIONS

Notation	Description
U_i	The user who request outsourcing services
S_i	The server who provide outsourcing services
T_i	The computation task
T_i^m	The m-th sub-task of computation task T_i
$T_i\_Deposit$	The deposit of the task T_i
C_{ij}	The contract signed by user U_i and server S_j
$C_{ij}\_Address$	The address of contract C_{ij}
$Sort()$	Sort function in ascending order
$Transfer()$	The function for transferring the deposit
gcd	The greatest common divisor
$F(x)$	The outsourcing function for computation task T_i
$f_i(x)$	The mathematical function for the i-th su-btask
X	The input to function $F(x)$
x_i	A portion of X
y_i	The computation result
C	The commitment to the function $F(x)$
ω_i	The proof of the computation result
I_i	The basis function
$p_i(x)$	A polynomial by Lagrange interpolation
$q_i(x)$	A quotient polynomial for a single proof
$q(x)$	A quotient polynomial for an aggregated proof

IV. PROPOSED FRAMEWORK

In this section, we provide a comprehensive explanation of the Libras and describe the three key components required for its construction: task allocation, task calculation and result verification. In the task allocation phase, after the user collects the computational tasks, multiple tasks are sorted from high to low according to the deposit and divided into smaller chunks by the Euclidean algorithm. The server selects the tasks according to its computational capacity. In the task computation phase, the server executes the computation tasks after receiving the functions and inputs, and uploads the proofs of the computation results to the blockchain. In the task verification phase, the verifier calculates the basis function of each server using the Lagrangian interpolation, and then verifies the computation results under the same task in batch by an aggregated proof.

In the Libras, it contains n users $U_1, U_2, \ldots, U_n$ who request outsourcing services, k servers $S_1, S_2, \ldots, S_k$ that provide outsourcing services, and some outsourcing computation tasks $T_1, T_2, \ldots, T_n$. Each computation task T_i contains five components, which are ID, $Deposit$, $Reward$, $Time$, and $Detail$. ID is used to identify the task. $Deposit$ refers to the advance payment that the server needs to make when selecting the task. $Reward$ is the bonus that the server earns upon successfully completing the computation task. $Time$ indicates the deadline for completing the computation task. And $Detail$ is the detailed description of the task.

The main notations used are listed in Table II.

A. Task Allocation

1) **Task collection.** The users initiate outsourcing computation tasks. These tasks are collected into a task pool. A task with high rewards implies that it is more computationally difficult and requires more computing resources. Therefore, servers with limited resources and weaker computing power have no advantage in selecting tasks and may even not be assigned tasks for a long period.

Authorized licensed use limited to: Institute of Information EngineeringCAS. Downloaded on March 28,2026 at 14:48:22 UTC from IEEE Xplore. Restrictions apply.

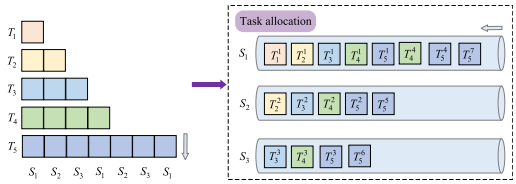


Fig. 2. Task allocation.

2) **Allocation policy.** To prevent servers with strong computing power from monopolizing tasks, leaving the weak servers with no tasks to do. We consider dividing the tasks into smaller chunks, allowing even weaker servers to have the opportunity to select and complete tasks. Firstly, we sort the tasks in ascending order based on their deposit and calculate the greatest common divisor (gcd) of all tasks' deposits by the Euclidean algorithm to determine the smallest unit size. Each task is divided into multiple sub-tasks based on the gcd. We simulate two types of outsourced computations, namely large matrix multiplication and modular exponentiation. For large matrix multiplication, to facilitate efficient calculation, the matrix can be divided into blocks either row-wise or column-wise. The product of these sub-matrices can be computed and the computation results are concatenated row-wise or column-wise, which ultimately obtains the overall product of the original large matrix. When dealing with modular exponentiation involving large numbers, the first step is to convert the exponent into its binary representation, followed by dividing the binary sequence into smaller, equally-sized sub-sequences. Subsequently, a modular exponentiation is performed for each sub-sequence, and the results are combined effectively using methods such as accumulated multiplication. Additionally, servers are sorted according to their computing power and select tasks in order. In each round of selection, the server only selects one sub-task of each task. Finally, after multiple rounds of selection, to avoid concentrating all computations on a single server, all servers sequentially select the remaining sub-tasks of the same task in turn.

As shown in Fig. 2, the order of the computing power of the servers is: $S_1 > S_2 > S_3$, and the order of the difficulty of the tasks is: $T_1 < T_2 < T_3 < T_4 < T_5$. According to the allocation policy, the size of the sub-task is the size of task T_1. S_1 selects the first sub-task of each T_i, S_2 selects the second sub-task of each T_i, and S_3 select the third sub-task of each T_i. After that, each server selects sub-tasks in turn again. When only sub-tasks of a large-scale task remain, such as Task T_5, if the server S_1 has already made a task selection, S_2 then selects the fifth sub-task within Task T_5 according to the order of computational capacities of the servers, followed by S_3 choosing the sixth sub-task of T_5, and ultimately, it returns to S_1's turn to pick the seventh sub-task within Task T_5. In summary, even when there remain only sub-tasks of a large-scale task, server S_i will still adhere to the sequence of its computational capacity in selecting sub-tasks.

3) **Task selection.** When the server selects a task, it needs to sign the task. At the same time, the server signs a smart contract C_{ij} with the user who initiates the task and transfers a deposit to the contract's designated address. The system determines the validity of the contract by checking whether the server has paid the deposit to the contract's designated address. If the validation is successful, the process of task allocation concludes.

Algorithm 1 Task Allocation Process

Input: Computing tasks T_i and the number of servers k

Output: True or False

```
Sort(T_i_Deposit);
gcd=GCD(T_i_Deposit);
for i = 1; i ≤ n; i++ do
    Num_{T_i} = T_i/gcd;
    for h = 1; h ≤ Num_{T_i}; h++ do
        Sig_{S_{h mod k}}(T_i^h);
        C_ij_Address ⟵ Transfer(T_i_Deposit);
        i++;
if C_ij_balance − C_ij_lastBalance == T_i_Deposit then
    return True;
else
    return False;
```

B. Task Calculation

1) Secret Sharing Based on Linear Algebra: The user firstly initiates a computational task T_i, denoted as $F(x) = a_0 + a_1x + a_2x^2 + \cdots + a_nx^n$. Let $(\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T)$ be a bi-linear group whose generators are $g_1, g_2, g_T = e(g_1, g_2)$ respectively. Next, the user chooses a random value $\tau \leftarrow F_q^*$, which is kept confidential. Using this privately chosen τ, the user proceeds to generate the public parameters $pp = (g_1, g_1^{\tau}, g_1^{\tau^2}, \ldots, g_1^{\tau^n})$. The commitment to the function $F(x)$ is $C = g_1^{F(\tau)} = g_1^{a_0 + a_1\tau + a_2\tau^2 + \cdots + a_n\tau^n} = \prod_{i=0}^{n}(g_1{}^{\tau^i})^{a_i}$. Subsequently, the user selects $b_1, b_2, \ldots, b_{t-1} \leftarrow F_q^*$, and constructs an arbitrary polynomial $B(u) = X + b_1u + b_2u^2 + \ldots + b_{t-1}u^{t-1}$. The secret value X is embedded as the constant term of the polynomial $B(u)$. The user arbitrarily takes t numbers $u_1, u_2, \ldots, u_t$, and brings them into the polynomial $B(u)$ to get t points $(u_1, B(u_1))$, $(u_2, B(u_2))$, $\ldots$, $(u_t, B(u_t))$. Finally, the user sends the t points to the servers $(S_1, S_2, \ldots, S_k)$.

2) Calculation Process: After receiving the values of these points from the user, servers reconstruct a polynomial $B'(u)$ using Lagrange interpolation and substitute $u = 0$ into the polynomial to obtain the secret value $X = B'(0)$. In addition, the computing function $F(x)$ is sent to each server in public. X is the input to the function $F(x)$ and $F(x)$ is the specific mathematical function of the computational task T_i. Assuming the computing task T_i is divided into k sub-tasks $(T_i^1, T_i^2, \ldots, T_i^k)$, the functions corresponding to the sub-tasks are denoted by $(f_1(x), f_2(x), \ldots, f_k(x))$, respectively. The function $F(x)$ concatenates the outputs of sub-tasks. That is $F(x) = f_1(x) \oplus f_2(x) \oplus \cdots \oplus f_k(x)$, where the symbol $\oplus$ represents different meanings depending on the type of computational function, including the addition of function values, the juxtaposition of vectors, and the concatenation of matrices, etc. For example, when there are two sets of secret input data $X = (x_1, x_2)$, x_i is a portion of X. For

Authorized licensed use limited to: Institute of Information EngineeringCAS. Downloaded on March 28,2026 at 14:48:22 UTC from IEEE Xplore. Restrictions apply.

Algorithm 2 Task Calculation Process

Input: The computing task T_i, the secret value X and servers $(S_1, S_2, \ldots, S_k)$

Output: The results of calculation (y_i, C, ω_i)

// User:

Choose $a_1, a_2, \ldots, a_n \leftarrow F_q^*$;

Set the computing task T_i: $F(x) = a_0 + a_1 x + a_2 x^2 + \ldots + a_n x^n$;

Fix generators g_1, g_2, and $g_T = e(g_1, g_2)$;

Choose $\tau \leftarrow F_q^*$;

Generate the public parameters $pp = (g_1, g_1^{\tau}, g_1^{\tau^2}, \ldots, g_1^{\tau^n})$;

$C = g_1^{F(\tau)}$;

Select $b_1, b_2, \ldots, b_n \leftarrow F_q^*$;

Construct an arbitrary polynomial $B(u) = X + b_1 u + b_2 u^2 + \ldots + b_n u^n$ ▷ Secretly transmit X;

Send $(u_i, B(u_i))_{i=1}^k$;

// Server:

Reconstruct X;

$y_i = F(x_i)$;

$z(x) = (x - x_1)(x - x_2) \ldots (x - x_k)$;

$I_i = \prod_{i \neq j}^{1<j<k} \frac{x - x_j}{x_i - x_j}$;

$p_i(x) = y_i I_i(x)$;

$q_i(x) = \frac{F(x)/k - p_i(x)}{z(x)} = q_0 + q_1 x + \cdots + q_{n-k} x^{n-k}$;

$\omega_i = g_1^{q_i(\tau)}$;

return (y_i, C, ω_i);

a specific server, after receiving the input x_i and function $F(x)$, it performs the computing task and obtains the output $y_i = F(x_i)$. When the input is $x = x_1$, the concatenated result is denoted as y_1 and can be calculated as $y_1 = F(x_1) = f_1(x_1) \oplus f_2(x_1) \oplus f_3(x_1) \oplus f_4(x_1)$. Similarly, upon substituting the input $x = x_2$ into the same function $F(x)$, the concatenated result is denoted as y_2 and can be calculated as $y_2 = F(x_2) = f_1(x_2) \oplus f_2(x_2) \oplus f_3(x_2) \oplus f_4(x_2)$. Given the transparent and open characteristics of blockchain, the computation results in plain text cannot be uploaded onto the chain. To safeguard the confidentiality of computation results, we upload the proofs of computation results generated by the commitment mechanism onto the chain.

Since each server performs computation independently and only has its computation results, without the computation results of other servers, we introduce Lagrangian interpolation based on basis functions to generate proofs for the subsequent verification. To prove an evaluation $F(x_i) = y_i$, the server firstly computes an accumulator polynomial $z(x) = (x - x_1)(x - x_2) \ldots (x - x_k)$ and the Lagrange basis function $I_i(x) = \prod_{i \neq j}^{1<j<k} \frac{x - x_j}{x_i - x_j}$ for the point (x_i, y_i). Then, it gets a polynomial $p_i(x) = y_i I_i(x)$ by Lagrange interpolation based on basis functions and computes a quotient polynomial $q_i(x) = \frac{F(x)/k - p_i(x)}{z(x)}$, where k is the number of sub-tasks. Let the coefficients of $q_i(x)$ be $(q_0, q_1, \ldots, q_{n-k})$. The proof ω_i is computed as $\omega_i = g_1^{q_i(\tau)} = g_1^{q_0 + q_1\tau + q_2\tau^2 + \cdots + q_{n-k}\tau^{n-k}} = \prod_{i=0}^{n-k} (g_1{}^{\tau^i})^{q_i}$.

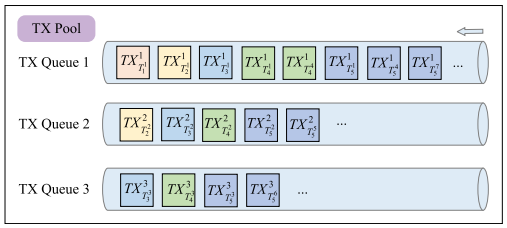


Fig. 3. TX pool.

3) Transaction Process: In this phase, the server constructs the blockchain transactions carrying the proofs of computation results. These transactions are put into a TX pool. We describe the DAG consensus algorithm in the following steps.

a) The blockchain nodes listen to the transactions in the network.

b) Transactions with valid signatures and free from double-spending are subsequently placed into the transaction (TX) pool.

c) The node uses an account-based system to sort these transactions in the TX pool. Specifically, each signed transaction contains a nonce. This nonce increments by one as a new transaction is processed. For instance, the nonce for the first transaction of a new account is one, and for the second transaction, the nonce is two. Subsequently, all transactions originating from the same sender form a transaction queue. As shown in Fig. 3, in the TX pool, transactions from the server S_1 constitute the first transaction queue, noted as TX Queue 1, transactions from the server S_2 form the second transaction queue, noted as TX Queue 2, and transactions from the server S_3 make up the third transaction queue, noted as TX Queue 3.

d) In the Bitcoin network, certain blocks, often referred to as orphaned or forked blocks, do not form part of the longest chain and are thus discarded. The transaction information in the orphaned blocks is not inherently erroneous, but they failed to join in the longest chain for some reason. Consequently, if some transactions are not packed into the longest valid chain, it will lead to the loss of computation results.

Inspired by the DAG-based blockchain, we design the ledger structure in the Libras. Specifically, the first transaction in each TX Queue is added to the ledger as the first block. In the same TX Queue, each block is linked to the previous block with a solid line. In different TX queues, but in the same task, each block is linked to the previous block with a dashed line. In other words, the solid line connects the computation results generated by a certain server, and the dashed line connects the computation results of a certain task. Moreover, when continuous transactions carry the computation results of the same task in the TX Queue, they are packed into the same block. For example, as shown in Fig. 4, each transaction is denoted as the form of $TX^k_{T_i^j}$, where i represents the task T_i, j represents the j-th sub-task of task T_i and k represents the server S_k. $TX^1_{T_1^1}$ is first added to the ledge. $TX^1_{T_2^1}$ points to $TX^1_{T_1^1}$ with a solid line, and $TX^1_{T_3^1}$ points to $TX^1_{T_2^1}$ with a solid line. $TX^2_{T_2^2}$ points to $TX^1_{T_2^1}$ with a dashed line because they both belong to the same task, but are calculated by different servers. $TX^1_{T_5^4}$ and $TX^1_{T_5^7}$ packed in the same block as $TX^1_{T_5^1}$

Authorized licensed use limited to: Institute of Information EngineeringCAS. Downloaded on March 28,2026 at 14:48:22 UTC from IEEE Xplore. Restrictions apply.

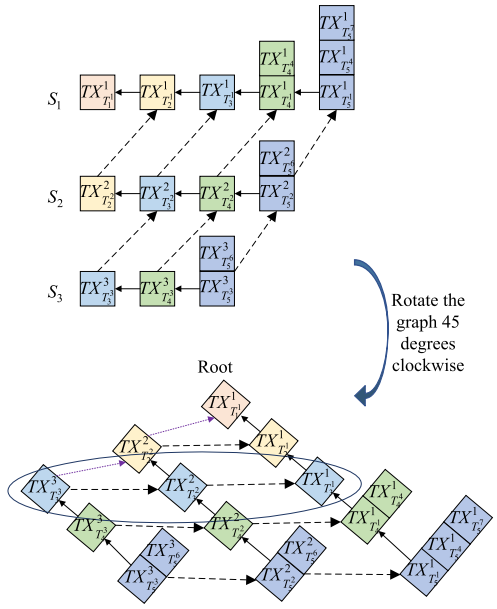


Fig. 4. The ledger structure in Libras.

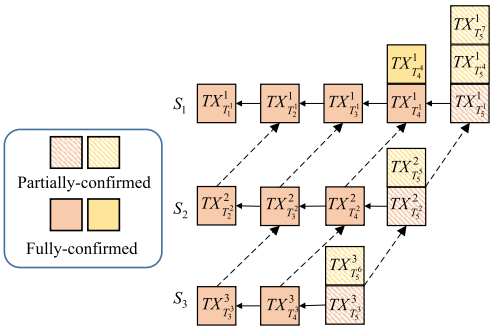


Fig. 5. Fully-confirmed blocks and partially-confirmed blocks.

Algorithm 3 Result Verification Process

Input: (C, ω_i)
Output: True or False
Aggregate Proofs $w = \prod_{i=1}^{k} \omega_i$;
if $e(C, g_2) == e(w, g_2^{z(x)}) g_T^{p(x)}$ **then**
  return $True$;
else
  return $False$;

because they are consecutive transactions in the same TX Queue for the same task.

e) Newly constructed blocks are published throughout the network, and all other nodes in the network can receive and execute the blocks.

C. Result Verification

From Section IV-B, we know that the transactions in the block carry the proofs of computation results. Our next step is to verify whether the server has accurately completed the task, leveraging these proofs. As shown in Fig. 4, by rotating the DAG-based ledger structure 45 degrees clockwise, we can transform it into a tree-like ledger structure. The genesis block in the DAG serves as the root of the tree. Utilizing a hierarchical traversal algorithm, we can extract the proofs of computation results in the same task. For instance, we can gain the proofs of the task T_3 by traversing the third level of the tree, which are ω_1, ω_2, ω_3 respectively.

1) Proofs Aggregation: To improve the efficiency of verification, the verifier aggregates multiple proofs to verify the computation results of all sub-tasks under the same task in batch. Suppose a task $F(x)$ has k sub-tasks, i.e., there are k points (x_1, y_1), (x_2, y_2), …, (x_k, y_k), as well as the corresponding k proofs $\omega_1, \omega_2, \ldots, \omega_k$. Let $q(x) = q_1(x) + q_2(x) + \cdots + q_k(x) = \frac{F(x) - \sum_{i=1}^{k} y_i I_i(x)}{z(x)}$ and $p(x) = \sum_{i=1}^{k} y_i I_i(x)$. The aggregated proof ω is computed as $\omega = \prod_{i=1}^{k} \omega_i = g_1^{q(\tau)} = g_1^{q_1(\tau)+q_2(\tau)+\cdots+q_k(\tau)}$.

The proofs can be verified using: $e(C, g_2) = e(w, g_2^{z(x)}) g_T^{p(x)}$. If the equation holds, it means that the computation results for all sub-tasks of the same task are correct. After these computation results are verified, the proofs of the correct computation results will be recorded in the blockchain's ledger structure. Their corresponding blocks will become fully confirmed blocks. The remaining blocks are partially confirmed blocks, as shown in Fig. 5.

Finally, servers that deliver correct computation results are marked as honest, while those that do not are identified as malicious. This also triggers the incentive and punishment mechanisms. All trustworthy servers are rewarded and their initial deposits are returned, while the deposits of the malicious servers are seized.

V. Performance Evaluation

In this section, we describe in detail the experimental setup and the experimental evaluation of the Libras. We measure and compare the time taken for task allocation, task calculation, and result verification under the operations of matrix multiplication and modular exponentiation, respectively. This comparison helps us to assess the performance of the Libras against other schemes. We denote the time for task allocation as $Setup$, the time for task calculation as $Prove$, and the time for result verification as $Verify$.

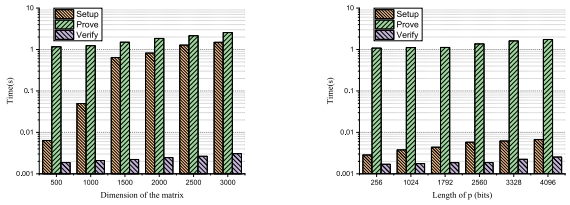
A. Experimental Setup

The experiments are implemented on Ubuntu 22.04 with Intel Core i7-12700U@1.80-GHz CPU, 4-GB RAM, 1-TB Hard Drive. The programming language is C++ with a GMP library and a PBC library. In outsourcing computation, complex operations such as matrix multiplication and modular exponentiation are often encountered and generally need to be delegated. We initialize 30 matrix multiplication tasks and 30 modular exponentiation tasks separately. The user outsources these tasks to the servers for calculation. We use the mainstream DAG-based system Conflux to build a blockchain network and run 20 Conflux nodes in Docker to simulate the edge server, i.e. task verifiers.

B. Experimental Results

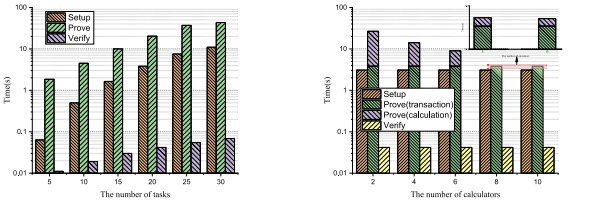
1) Execution Time for Each Phase: For the task of matrix multiplication, we carry out experiments on different matrix

Authorized licensed use limited to: Institute of Information EngineeringCAS. Downloaded on March 28,2026 at 14:48:22 UTC from IEEE Xplore. Restrictions apply.



(a) Matrix multiplication (b) Modular exponentiation

Fig. 6. Execution time for each phase.



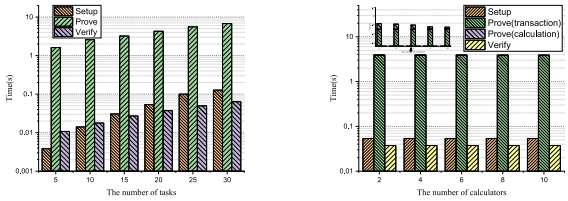
(a) Execution time under different numbers of tasks (b) Execution time under different numbers of task calculators

Fig. 7. Matrix multiplication.

dimensions and record the execution time at each phase. Fig. 6(a) shows that as the dimensions of the matrix enlarge, the time for task allocation, task calculation, and result verification gradually increases, with the majority of the time being spent on the task calculation stage. In addition, the time for task verification exhibits the least fluctuates, indicating that the increase in matrix dimensions has a smaller impact on the time required for task verification.

With a fixed number of task calculators, we measure the execution time for different numbers of tasks in each phase. As shown in Fig. 7(a), we can observe that as the quantity of tasks grows, there is a corresponding increase in the time required for task allocation, task calculation, and result verification. The time spent on task allocation and task calculation significantly outweighs that spent on task verification, consistent with our theoretical analysis. During the task allocation phase, which includes task collection, allocation policy, and task selection, the task calculator needs to pay a deposit and sign for the selected task, thereby making task selection a time-intensive process. During the task calculation phase, the computation result of each sub-task requires a proof, as well as a blockchain transaction, making this the most time-consuming stage.

With a fixed number of matrix multiplication tasks, we measure the execution time for task allocation, task calculation, and result verification involving varying quantities of task calculators. As depicted in Fig. 7(b), the task allocation time and the result verification time remain relatively constant as the quantity of task calculators increases. This is because the time for task allocation and result verification is only related to the quantity of tasks rather than the number of calculators. Overall, the task calculation time exhibits a decreasing trend with the growth in the quantity of task calculators. Specifically, the task calculation time comprises both the time spent on task execution and transaction transmission, namely, the calculation of the results and the uploading of the results via blockchain transactions, where the time taken for task execution gradually decreases, whereas the time for transaction transmission remains largely unchanged. This occurs because the former component is directly related to the number of task calculators.



(a) Execution time under different numbers of tasks (b) Execution time under different numbers of task calculators

Fig. 8. Modular exponentiation.

As more calculators participate in the calculations, the task calculation time progressively shortens. On the other hand, the latter component is unrelated to the number of task calculators but rather depends on the number of tasks. When the number of tasks is fixed, the time for transaction transmission stays essentially constant. To vividly emphasize this phenomenon, we have magnified the task calculation times when there are 8 and 10 calculators. When the time spent on task execution is minimal, a significant portion of the total time is devoted to transaction transmission.

For large number modular exponentiation tasks of the form $x^a \bmod p$, we implement experiments with different lengths of exponent p and record the execution time of different phases. Fig. 6(b) shows as the length of the exponent p increases, the execution time of each phase also gradually increases. The task calculation phase consumes a notably greater amount of time compared to both the task allocation and result verification phases. Furthermore, the time expended on task verification exhibits the most stability, suggesting that the increment in the length of the exponent p exerts a relatively minor influence on the time required for task verification.

Fig. 8(a) illustrates the execution time of the task allocation, the task calculation, and the result verification for different numbers of tasks with a fixed number of task calculators. It demonstrates that as the count of tasks escalates, the duration for each phase also expands. In addition, during the task allocation phase, the system ranks the tasks from easiest to most challenging based on the deposit. The more tasks there are, the longer the task allocation time. The more difficult the task, the longer it takes to compute. Consequently, both the task allocation duration and task calculation duration have risen significantly.

As shown in Fig. 8(b), with a fixed number of modular exponentiation tasks, we measure the time for task allocation, the task calculation, and the result verification under different numbers of task calculators. In Fig. 8(b), the trends exhibited across various stages are consistent with those seen in Fig. 7(b), along with the underlying reasons contributing to these trends. By magnifying the task calculation duration, it becomes evident that while the number of calculators increases, the time required to upload results via blockchain transactions remains unchanged. This is because the duration of transaction transmission is solely determined by the number of tasks, not the number of calculators. Simultaneously, it is also discernible that as the quantity of calculators rises, the duration expended on calculating the results gradually decreases. Essentially, the more calculators engaged in the calculation process, the less time is needed to complete the computations.

Authorized licensed use limited to: Institute of Information EngineeringCAS. Downloaded on March 28,2026 at 14:48:22 UTC from IEEE Xplore. Restrictions apply.

In our proposed scheme, both the task difficulty and server capability are arranged in order. Additionally, tasks are divided into smaller sub-tasks, enabling servers with lower computational power to select less challenging sub-tasks. As evidenced in experimental illustrations, servers consistently prioritize and execute simpler tasks initially, ensuring that each server, even with weak computing power, has a share of sub-tasks to perform. Furthermore, utilizing smart contracts in the blockchain, servers are required to sign a contract and pay a deposit before selecting tasks. Servers that compute correctly are rewarded, while malicious servers are penalized by forfeiting their deposits. Given the foregoing analysis, our scheme can guarantee fairness.

In the Libras, we use a DAG-based blockchain to design the outsourcing scheme. Unlike the conventional blockchain, a DAG-based blockchain accommodates all transactions and blocks without data loss. As depicted in the green part of Fig. 7(b) and Fig. 8(b), with the same number of tasks, the time consumed for blockchain transactions remains consistent, indicating that all transactions carrying computation results can be recorded in the blockchain without exception. Thereby, the risk of transactions going unrecorded and consequently resulting in data loss is eliminated. Each transaction is directly involved in maintaining the order of transactions across the network, collectively forming the structure of a graph. Upon initiation, transactions are broadcast directly to the whole network, thus increasing the speed of transaction processing which can provide fast transaction confirmation and elastic scalability.

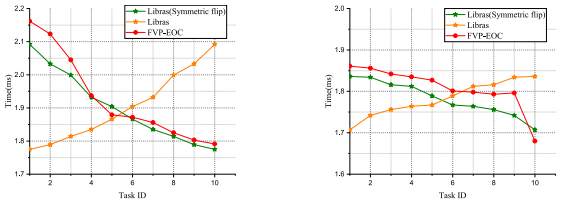
2) Performance Comparison: According to the comparison in Table I, Dredas [32] and BPay proposed by Zhang et al. [7] are applied in outsourcing storage services, while the study by Huang et al. [33] and OBFP [34] respectively deal with the task type of SHA-256 hash inversion and outsourcing offline dictionary guessing. Due to the inherently diverse nature of these applications, a direct quantitative comparison with Libras is not feasible. Whereas, FVP-EOC [14] has a similar architecture to Libras and is a typical blockchain-based outsourcing computing solution that simulates the outsourcing computing of matrix multiplication and modular exponentiation. Here, we choose to analyze a quantitative comparison with FVP-EOC.

In the task allocation phase, the two schemes are similar in that they both require task sorting, task division, and task selection. In the task calculation process, FVP-EOC utilizes ElGamal encryption algorithms to calculate the ciphertext of the computation results by $Cip_i = (C_1, C_2) = (g^k \bmod P, y_i^{k_i} \cdot m_i \bmod P)$. For a single computation result, this process involves a modular exponentiation operation and a modular multiplication operation after an exponentiation operation. However, our scheme does not require the encryption and decryption operations, but rather the commitment $C = g_1^{F(\tau)}$ and proofs $\omega_i = g_1^{q_i(\tau)}$ of the computation results. Assuming there are n results, it takes time $n(T_{me}+T_e+T_{mm})$ in FVP-EOC and $(n+1)T_e$ in our scheme. Obviously, the time of task calculation in FVP-EOC is longer than ours. In the result verification phase, FVP-EOC utilizes the similarity of results to verify and decrypt the encrypted results. The time in this phase is mainly in decryption, which requires n modular exponentiation operations for n encrypted results. While ours only needs to verify whether two bi-linear pairings are equal to batch verify the computation results. The computational overhead for each phase is listed in Table III.

TABLE III
COMPARISON OF COMPUTATIONAL OVERHEAD

Scheme	FVP-EOC [14]	Libras
Task allocation[1]	$T_{sor}+T_{div}+T_{sel}$	$T_{sor}+T_{div}+T_{sel}$
Task calculation[2]	$n(T_{me}+T_e+T_{mm})$	$(n+1)T_e$
Result verification[3]	nT_{me}	$2T_p$

[1] T_{sor}, T_{div}, and T_{sel} denote the time for sorting tasks, dividing tasks, and selecting tasks, respectively.
[2] T_{me}, T_e, and T_{mm} represent the time for modular exponentiation, exponentiation, and modular multiplication, respectively.
[3] T_p indicates the time for bilinear pairing.



(a) matrix multiplication (b) Modular exponentiation

Fig. 9. The comparison of the verification time.

Given an equivalent count of task calculators, we compare the verification durations for matrix multiplication tasks and modular exponentiation tasks between Libras and FVP-EOC, as shown in Fig. 9. We record the verification time corresponding to different task IDs and plot these points on a graph where each task ID serves as the abscissa and its associated verification time forms the ordinate. Subsequently, employing these plotted points, we interpolate the trend curves and compute the ratio of their respective slopes.

In FVP-EOC, the tasks are executed in an order that transitions from hard to easy as the task ID increases. Conversely, in Libras, the tasks are executed from easy to hard. To facilitate a more intuitive comparison of the two schemes, we symmetrically flipped Libras' verification time curve at the task ID of 5.5, transforming it from the original orange curve to the green curve depicted in Fig. 9(a). We can see that the harder the task, the more verification time it takes. The verification time in Libras is shorter than in FVP-EOC. The slope of the curve illustrates that as the difficulty of the task increases, the verification time in Libras is significantly less than in FVP-EOC. This efficiency is attributed to Libras' use of proofs aggregation during the verification of computation results, which enables simultaneous batch verification of results from multiple sub-tasks, thereby reducing the verification time. In the Libras, the calculated slope for verification time stands at -0.29. Comparatively, in the FVP-EOC, the slope for verification time measures at -0.35. The ratio between these slopes is approximately 1.2. Therefore, the slope reveals that Libras is approximately 1.2× that of FVP-EOC in verification duration. Similarly, we compare the verification duration of modular exponentiation tasks, given an equivalent quantity of task calculators between Libras and FVP-EOC, as shown in Fig. 9(b). To compare the two schemes more visually, we symmetrically flipped Libras' verification time curve at

Authorized licensed use limited to: Institute of Information EngineeringCAS. Downloaded on March 28,2026 at 14:48:22 UTC from IEEE Xplore. Restrictions apply.

the task ID of 5.5, changing it from the orange curve to the green curve. Fig. 9(b) also shows that the verification time in Libras is generally less than in FVP-EOC.

VI. Security Analysis

We give the proof in this section to prove the security goals of Section III-D.

A. Security

Security requires that the secret input X and the computation results y_i are not leaked out throughout the complete process of transmission and verification. In the transmission process, the security of X and y_i is dependent on secret sharing. We consider the worst-case scenario and assume that the adversary obtains t-1 points $(u_i, B(u_i))_{i=1}^{t-1}$. We give the following theorem.

Theorem 1. *If there are at most t-1 servers colluding in this scenario, then any set of servers less than t cannot recover the secret input X and the computation results y_i.*

Proof: According to the description of Libras in this paper, the user chooses $b_1, b_2, \ldots, b_n \leftarrow F_q^*$, and constructs an arbitrary polynomial $B(u) = X + b_1 u + b_2 u^2 + \ldots + b_n u^n$. Then it selects t points $(u_i, B(u_i))_{i=1}^{t}$ on polynomial $B(u)$, and sends $(u_i, B(u_i))_{i=1}^{t}$ to the servers. Assume that there are $t-1$ malicious servers, they can construct the following system of equations.

$$\begin{cases} b_0 + b_1 u_1^1 + \cdots + b_{t-1} u_1^{t-1} = B(u_1) \\ b_0 + b_1 u_2^1 + \cdots + b_{t-1} u_2^{t-1} = B(u_2) \\ \cdots \\ b_0 + b_1 u_t^1 + \cdots + b_{t-1} u_t^{t-1} = B(u_t) \end{cases} \tag{1}$$

$$\begin{pmatrix} 1 & \cdots & u_1^{t-1} \\ \vdots & \ddots & \vdots \\ 1 & \cdots & u_t^{t-1} \end{pmatrix} \begin{bmatrix} b_0 \\ \cdots \\ b_t \end{bmatrix} = \begin{bmatrix} B(u_1) \\ \cdots \\ B(u_t) \end{bmatrix} \tag{2}$$

$$\begin{vmatrix} 1 & \cdots & u_1^{t-1} \\ \vdots & \ddots & \vdots \\ 0 & \cdots & 0 \\ \vdots & \ddots & \vdots \\ 1 & \cdots & u_t^{t-1} \end{vmatrix} = 0 \tag{3}$$

$t-1$ malicious servers can construct $t-1$ linearly independent equations by these points. They need to solve the unique solution of the matrix equation if they want to get the secret data $X = b_0$. But now $t-1$ malicious servers have only $t-1$ linearly independent equations, which means that the data in one row of the coefficient matrix of the matrix equation must all be zero. The determinant of the coefficient matrix is equal to 0, as in (3), i.e., there is no unique non-zero solution to this matrix equation. Therefore, $t-1$ malicious servers have no access to the secret input X and the computation result y_i. □

In the verification process, the security of computation results y_i relies on the complexity of resolving the elliptic curve discrete logarithm problem (ECDLP) over a finite field.

Theorem 2. *Under the ECDLP, the Libras is secure. For $\epsilon > 0$, our scheme is said to be ϵ-secure if for any input X, any computation results y_i, and any adversary $\mathcal{A}$, $Pr[\mathrm{EXP}_{\mathcal{A},\Pi} = 1] \leq \epsilon$.*

Proof: If a PPT adversary $\mathcal{A}$ wants to obtain the computation results y_i during the verification process, it should be able to tackle the ECDLP over a finite field. Given the (x, y), it is difficult to compute $x \in \mathbb{Z}_p$ satisfying $y = g^x \bmod p$. The probability of successfully computing x for $\mathcal{A}$ is ϵ, which has been proven to be negligible. Therefore, the Libras is ϵ-secure in verification process. □

B. Verifiability

Theorem 3. *In the Libras, the verifier can accurately verify the accuracy of computation results in the result verification phase.*

Proof: Once the user receives the computation results from each server, which operates independently and possesses only its outcome, the user proceeds to compute k basis functions for these computation results by $I_i = \prod_{i \neq j}^{1<j<k} \frac{x - x_j}{x_i - x_j}$, where $i = (1, 2, \ldots, k)$ and k denotes the number of subtasks. The user then gets a polynomial $p(x) = \sum_{i=1}^{k} y_i I_i(x)$ by Lagrange interpolation based on basis functions. Finally, the user computes $g_T^{p(\tau)}$ and sends it to the verifier for verification. We know that the commitment to a function $F(x)$ is $C = g_1^{F(\tau)}$ and the proof of a computation result is $\omega_i = g_1^{q_i(\tau)} = g_1^{\frac{F(\tau)/k - y_i I_i(\tau)}{z(\tau)}}$. The k proofs generated by the k servers are $w_1 = g_1^{\frac{F(\tau)/k - y_1 I_1(\tau)}{z(\tau)}}$, $w_2 = g_1^{\frac{F(\tau)/k - y_2 I_2(\tau)}{z(\tau)}}$, $\ldots$, $w_k = g_1^{\frac{F(\tau)/k - y_k I_k(\tau)}{z(\tau)}}$, respectively.

After the verifier obtains the proofs $(\omega_1, \omega_2, \ldots, \omega_k)$ of all the computation results from servers, it performs the proofs aggregation $\omega = \prod_{i=0}^{k} \omega_i = g_1^{q(\tau)} = g_1^{q_1(\tau) + q_2(\tau) + \cdots + q_k(\tau)}$. The aggregated proof can be verified to satisfy the condition:

$$e(C, g_2) = e(\omega, g_2^{z(x)}) g_T^{p(x)}. \tag{4}$$

Substituting $C = g_1^{F(\tau)}$ and $\omega = g_1^{q(\tau)}$ into Equation 4, the subsequent derivation proceeds as follows.

$$e(g_1^{F(\tau)}, g_2) = e(g_1^{q(\tau)}, g_2^{z(\tau)}) g_T^{p(\tau)} \tag{5}$$

$$e(g_1, g_2)^{F(\tau)} = e(g_1, g_2)^{q(\tau) z(\tau)} g_T^{p(\tau)} \tag{6}$$

$$e(g_1, g_2)^{F(\tau)} = e(g_1, g_2)^{q(\tau) z(\tau) + p(\tau)} \tag{7}$$

$$F(\tau) = q(\tau) z(\tau) + p(\tau) \tag{8}$$

$$F(\tau) - p(\tau) = q(\tau) z(\tau) \tag{9}$$

The idea of verification is that the verifier utilizes a known point $(\tau, F(\tau))$ on a function $F(x)$ to verify that the point (x_i, y_i) is also on the function $F(x)$. We can see that Equation 9 holds at $x = \tau$, which means that the correctness of the computation results can be proved by the commitment and proofs.

If the server does not perform honest calculations, and generates the proof w_i^* based on random y_i^*. The dishonest server will calculate a wrong quotient polynomial $q_i(\tau)^* = \frac{F(\tau)/k - y_i^* I_i(\tau)}{z(\tau)}$ based on the wrong y_i^*. Since $F(\tau)/k - y_i^* I_i(\tau)$ is indivisible by $z(\tau)$, meaning that dividing $F(\tau)/k - y_i^* I_i(\tau)$ by $z(\tau)$ will always yield a remainder, this consequently leads

Authorized licensed use limited to: Institute of Information EngineeringCAS. Downloaded on March 28,2026 at 14:48:22 UTC from IEEE Xplore. Restrictions apply.

to $q_i(\tau)^* \neq q_i(\tau)$. Considering that $q(\tau)^*$ is derived from the sum $q(\tau)^* = q_1(\tau) + q_2(\tau) + \cdots + q_i(\tau)^*$, it logically follows that $q(\tau)^* \neq q(\tau)$. Therefore, $F(\tau) - p(\tau) = q(\tau)^* z(\tau)$ does not hold and (y_i^*, C, w_i^*) cannot pass the verification.

We can also verify an individual proof by Equation 4. A verifier who has a computation result (x_i, y_i), the commitment C and a proof ω_i checks $q_i(x) = \frac{F(x)/k - p_i(x)}{z(x)}$ holds at $x = \tau$ and $k = 1$ using Equation 4, where $z(x) = (\tau - x_i)$ and $p(x) = p_i(x) = y_i$. Substituting $C = g_1^{F(\tau)}$ and $\omega_i = g_1^{q_i(\tau)}$ into Equation 4, the subsequent derivation proceeds as follows.

$$e(g_1^{F(\tau)}, g_2) = e(g_1^{q_i(\tau)}, g_2^{z(\tau)}) g_T^{y_i} \tag{10}$$

$$e(g_1, g_2)^{F(\tau)} = e(g_1, g_2)^{q_i(\tau) z(\tau)} g_T^{y_i} \tag{11}$$

$$e(g_1, g_2)^{F(\tau)} = e(g_1, g_2)^{q_i(\tau)(\tau - x_i) + y_i} \tag{12}$$

$$F(\tau) = q_i(\tau)(\tau - x_i) + y_i \tag{13}$$

$$F(\tau) - y_i = q_i(\tau)(\tau - x_i) \tag{14}$$

Likewise, if the server does not perform honest calculations, and generates the proof w_i^* based on random y_i^*. The dishonest server will calculate a wrong quotient polynomial $q_i(\tau)^* = \frac{F(\tau)/k - p_i(\tau)}{z(\tau)} = \frac{F(\tau) - y_i^*}{\tau - x_i}$ based on the wrong y_i^*. Since that $F(\tau) - y_i^*$ is not divisible by $\tau - x_i$, which implies that dividing $F(\tau) - y_i^*$ by $\tau - x_i$ will always yield a remainder, this consequently leads to $q_i(\tau)^* \neq q_i(\tau)$. Therefore, $F(\tau) - y_i = q_i(\tau)^*(\tau - x_i)$ does not hold and (y_i^*, C, w_i^*) cannot pass the verification. □

VII. Conclusion

This paper proposed the Libras: a fair, secure, verifiable and scalable outsourcing computation scheme based on blockchain. We devised an easy-to-operate and resource-efficient task allocation strategy to maintain fairness. In addition, we integrated a commitment mechanism with on-chain and off-chain collaboration which stores computation results off-chain and store proofs on-chain for security and privacy. Moreover, we used a DAG-based blockchain to improve the scalability and reduce transaction latency. To guarantee the accuracy of the outsourcing computation results, we designed an efficient and reliable batch verification algorithm. Finally, theoretical analysis and experimental results demonstrated that the Libras is fair, secure, verifiable, and scalable. The comparison results indicate that the time of verification is 1.2× that of FVP-EOC. In the future, we are dedicated to researching more efficient and publicly verifiable blockchain-based outsourcing solutions and designing a generic verification algorithm for different types of computing tasks.

References

[1] Y. He, Y. Wang, C. Qiu, Q. Lin, J. Li, and Z. Ming, "Blockchain-based edge computing resource allocation in IoT: A deep reinforcement learning approach," *IEEE Internet Things J.*, vol. 8, no. 4, pp. 2226–2237, Feb. 2021.

[2] A. A. Abdellatif et al., "MEdge-chain: Leveraging edge computing and blockchain for efficient medical data exchange," *IEEE Internet Things J.*, vol. 8, no. 21, pp. 15762–15775, Nov. 2021.

[3] M. B. Mollah et al., "Blockchain for the Internet of Vehicles towards intelligent transportation systems: A survey," *IEEE Internet Things J.*, vol. 8, no. 6, pp. 4157–4185, Mar. 2021.

[4] K. Gai, Y. Wu, L. Zhu, L. Xu, and Y. Zhang, "Permissioned blockchain and edge computing empowered privacy-preserving smart grid networks," *IEEE Internet Things J.*, vol. 6, no. 5, pp. 7992–8004, Oct. 2019.

[5] Y. Wu, H.-N. Dai, and H. Wang, "Convergence of blockchain and edge computing for secure and scalable IIoT critical infrastructures in industry 4.0," *IEEE Internet Things J.*, vol. 8, no. 4, pp. 2300–2317, Feb. 2021.

[6] H. Zhang, L. Tong, J. Yu, and J. Lin, "Blockchain-aided privacy-preserving outsourcing algorithms of bilinear pairings for Internet of Things devices," *IEEE Internet Things J.*, vol. 8, no. 20, pp. 15596–15607, Oct. 2021.

[7] Y. Zhang, R. H. Deng, X. Liu, and D. Zheng, "Outsourcing service fair payment based on blockchain and its applications in cloud computing," *IEEE Trans. Services Comput.*, vol. 14, no. 4, pp. 1152–1166, Jul./Aug. 2021.

[8] H. Zhang, P. Gao, J. Yu, J. Lin, and N. N. Xiong, "Machine learning on cloud with blockchain: A secure, verifiable and fair approach to outsource the linear regression," *IEEE Trans. Netw. Sci. Eng.*, vol. 9, no. 6, pp. 3956–3967, Nov. 2022.

[9] R. Yang, F. R. Yu, P. Si, Z. Yang, and Y. Zhang, "Integrated blockchain and edge computing systems: A survey, some research issues and challenges," *IEEE Commun. Surveys Tuts.*, vol. 21, no. 2, pp. 1508–1532, 2nd Quart., 2019.

[10] K. Gai, J. Guo, L. Zhu, and S. Yu, "Blockchain meets cloud computing: A survey," *IEEE Commun. Surveys Tuts.*, vol. 22, no. 3, pp. 2009–2030, 3rd Quart., 2020.

[11] H. Xue, D. Chen, N. Zhang, H.-N. Dai, and K. Yu, "Integration of blockchain and edge computing in Internet of Things: A survey," *Future Gener. Comput. Syst.*, vol. 144, pp. 307–326, 2023.

[12] L. Yuan et al., "CoopEdge: A decentralized blockchain-based platform for cooperative edge computing," in *Proc. Web Conf.*, pp. 2245–2257, 2021.

[13] C. Li et al., "A decentralized blockchain with high throughput and fast confirmation," in *Proc. USENIX Annu. Tech. Conf. (USENIX ATC)*, 2020, pp. 515–528.

[14] T. Li, Y. Tian, J. Xiong, and M. Z. A. Bhuiyan, "FVP-EOC: Fair, verifiable, and privacy-preserving edge outsourcing computing in 5G-enabled IIoT," *IEEE Trans. Ind. Informat.*, vol. 19, no. 1, pp. 940–950, Jan. 2023.

[15] S. Zhang, H. Li, Y. Dai, J. Li, M. He, and R. Lu, "Verifiable outsourcing computation for matrix multiplication with improved efficiency and applicability," *IEEE Internet Things J.*, vol. 5, no. 6, pp. 5076–5088, Dec. 2018.

[16] C. Liu, X. Hu, X. Chen, J. Wei, and W. Liu, "SDIM: A subtly designed invertible matrix for enhanced privacy-preserving outsourcing matrix multiplication and related tasks," *IEEE Trans. Dependable Secure Comput.*, 2023.

[17] K. Zhou, M. H. Afifi, and J. Ren, "ExpSOS: Secure and verifiable outsourcing of exponentiation operations for mobile cloud computing," *IEEE Trans. Inf. Forensics Security*, vol. 12, no. 11, pp. 2518–2531, Nov. 2017.

[18] T. Zhang and J. Wang, "Secure outsourcing algorithms of modular exponentiations in edge computing," in *Proc. IEEE 19th Int. Conf. Trust, Secur. Privacy Comput. Commun. (TrustCom)*, Dec. 2020, pp. 576–583.

[19] H. Li, J. Yu, H. Zhang, M. Yang, and H. Wang, "Privacy-preserving and distributed algorithms for modular exponentiation in IoT with edge computing assistance," *IEEE Internet Things J.*, vol. 7, no. 9, pp. 8769–8779, Sep. 2020.

[20] W. Song, B. Wang, Q. Wang, C. Shi, W. Lou, and Z. Peng, "Publicly verifiable computation of polynomials over outsourced data with multiple sources," *IEEE Trans. Inf. Forensics Security*, vol. 12, no. 10, pp. 2334–2347, Oct. 2017.

[21] J. Zhou, K.-K. R. Choo, Z. Cao, and X. Dong, "PVOPM: Verifiable privacy-preserving pattern matching with efficient outsourcing in the malicious setting," *IEEE Trans. Dependable Secure Comput.*, vol. 18, no. 5, pp. 2253–2270, Sep./Oct. 2021.

[22] J. Zhang, T. Jiang, X. Gao, and G. Chen, "An online fairness-aware task planning approach for spatial crowdsourcing," *IEEE Trans. Mobile Comput.*, vol. 23, no. 1, pp. 150–163, Jan. 2024.

[23] D. Li et al., "Decentralized IoT resource monitoring and scheduling framework based on blockchain," *IEEE Internet Things J.*, vol. 10, no. 24, pp. 21135–21142, Dec. 2023.

[24] L. Yin, J. Sun, J. Zhou, Z. Gu, and K. Li, "ECFA: An efficient convergent firefly algorithm for solving task scheduling problems in cloud-edge computing," *IEEE Trans. Services Comput.*, vol. 16, no. 5, pp. 3280–3293, Sep./Oct. 2023.

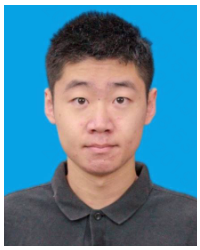
Authorized licensed use limited to: Institute of Information EngineeringCAS. Downloaded on March 28,2026 at 14:48:22 UTC from IEEE Xplore. Restrictions apply.

[25] X. Cai et al., "OVERSEE: Outsourcing verification to enable resource sharing in edge environment," in *Proc. 49th Int. Conf. Parallel Process.*, 2020, pp. 1–11.
[26] Z. Chen, A. Fu, R. H. Deng, X. Liu, Y. Yang, and Y. Zhang, "Secure and verifiable outsourced data dimension reduction on dynamic data," *Inf. Sci.*, vol. 573, pp. 182–193, Sep. 2021.
[27] H. Li, J. Yu, J. Fan, and Y. Pi, "DSOS: A distributed secure outsourcing system for edge computing service in IoT," *IEEE Trans. Syst., Man, Cybern., Syst.*, vol. 53, no. 1, pp. 238–250, Jan. 2023.
[28] H. Cui, Z. Wan, X. Wei, S. Nepal, and X. Yi, "Pay as you decrypt: Decryption outsourcing for functional encryption using blockchain," *IEEE Trans. Inf. Forensics Security*, vol. 15, pp. 3227–3238, 2020.
[29] C. Lin, D. He, X. Huang, X. Xie, and K.-K. R. Choo, "Blockchain-based system for secure outsourcing of bilinear pairings," *Inf. Sci.*, vol. 527, pp. 590–601, Jul. 2020.
[30] Y. Guan, H. Zheng, J. Shao, R. Lu, and G. Wei, "Fair outsourcing polynomial computation based on the blockchain," *IEEE Trans. Services Comput.*, vol. 15, no. 5, pp. 2795–2808, Sep. 2022.
[31] Z. Chen, Y. Tian, J. Xiong, C. Peng, and J. Ma, "Towards reducing delegation overhead in replication-based verification: An incentive-compatible rational delegation computing scheme," *Inf. Sci.*, vol. 568, pp. 286–316, Aug. 2021.
[32] K. Fan, Z. Bao, M. Liu, A. V. Vasilakos, and W. Shi, "Dredas: Decentralized, reliable and efficient remote outsourced data auditing scheme with blockchain smart contract for industrial IoT," *Future Gener. Comput. Syst.*, vol. 110, pp. 665–674, Sep. 2020.
[33] H. Huang, X. Chen, Q. Wu, X. Huang, and J. Shen, "Bitcoin-based fair payments for outsourcing computations of fog devices," *Future Gener. Comput. Syst.*, vol. 78, pp. 850–858, Jan. 2018.
[34] C. Lin, D. He, X. Huang, and K.-K. R. Choo, "OBFP: Optimized blockchain-based fair payment for outsourcing computations in cloud computing," *IEEE Trans. Inf. Forensics Security*, vol. 16, pp. 3241–3253, 2021.
[35] L. F. Zhang and H. Wang, "Multi-server verifiable computation of low-degree polynomials," in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2022, pp. 596–613.
[36] A. Chandramouli, A. Choudhury, and A. Patra, "A survey on perfectly secure verifiable secret-sharing," *ACM Comput. Surv.*, vol. 54, no. 11, pp. 1–36, 2022.
[37] S. Srinivasan, A. Chepurnoy, C. Papamanthou, A. Tomescu, and Y. Zhang, "Hyperproofs: Aggregating and maintaining proofs in vector commitments," in *Proc. 31st USENIX Secur. Symp. (USENIX Secur.)*, 2022, pp. 3001–3018.

Lijuan Huo received the B.S. degree from the Department of Computer, North China Electric Power University, Baoding, China, in 2019, and the M.S. degree from the School of Cyberspace Security, Zhengzhou University, Zhengzhou, China, in 2022. She is currently pursuing the Ph.D. degree with the School of Cyber Science and Engineering, Wuhan University, Wuhan, China. Her research interests include data security and verifiable computation.

Libing Wu received the B.S. and M.S. degrees in computer science from Central China Normal University, Wuhan, China, in 1994 and 2001, respectively, and the Ph.D. degree from Wuhan University, Wuhan, in 2006. He is currently a Professor with the School of Cyber Science and Engineering, Wuhan University. He is also a Researcher with Guangdong Laboratory of Artificial Intelligence and Digital Economy (SZ), Shenzhen, China. His research interests include network security, the Internet of Things, machine learning, and data security.

Zhuangzhuang Zhang received the B.S. degree in software engineering from Taiyuan University of Technology, Taiyuan, China, in 2017, and the M.S. degree from the College of Information and Computer, Taiyuan University of Technology, in 2020. He is currently pursuing the Ph.D. degree with the School of Cyber Science and Engineering, Wuhan University, China. His main research interests include AI security and data security.



Chunshuo Li received the B.S. degree from the School of Cyber Science and Engineering, Wuhan University, Wuhan, China, in 2022, where he is currently pursuing the master's degree. His research interests include verifiable privacy computing.

Debiao He (Member, IEEE) received the Ph.D. degree in applied mathematics from the School of Mathematics and Statistics, Wuhan University, Wuhan, China, in 2009. He is currently a Professor with the School of Cyber Science and Engineering, Wuhan University. He has authored or coauthored more than 100 research papers in refereed international journals and conferences, such as IEEE TRANSACTIONS ON DEPENDABLE AND SECURE COMPUTING, IEEE TRANSACTIONS ON INFORMATION FORENSICS AND SECURITY, and Usenix Security Symposium. His work has been cited more than 10000 times on Google Scholar. His main research interests include cryptography and information security, in particular, cryptographic protocols. He was a recipient of the 2018 IEEE SYSTEMS JOURNAL Best Paper Award and the 2019 *IET Information Security* Best Paper Award. He is on the Editorial Board of several international journals, such as *ACM Distributed Ledger Technologies: Research and Practice* and *Frontiers of Computer Science*.

Jing Wang received the Ph.D. degree in computer science from Wuhan University, Wuhan, China, in 2021. She is currently a Lecturer with the School of Cyber Science and Engineering, Huazhong University of Science and Technology, Wuhan. Her main research interests include cryptography and secure multi-party computation, in particular, secure cloud storage and cryptographic protocols.

Authorized licensed use limited to: Institute of Information EngineeringCAS. Downloaded on March 28,2026 at 14:48:22 UTC from IEEE Xplore. Restrictions apply.